



دانشگاه تهران

دانشکده مهندسی برق و کامپیوتر

درس شبکه‌های عصبی و یادگیری عمیق
پاسخ تمرین شماره پنج

دانشجویان:

سوال اول: رضا مرید شوشتری ۸۱۰۱۰۴۲۵۰

سوال دوم: علی رجب زاده ۸۱۰۱۰۴۱۳۷

فهرست مطالب

۴	فهرست تصاویر
۵	فهرست جداول
۶	۱ بازشناسی تصاویر با استفاده از ترنسفورمر
۶	۱-۱ آماده‌سازی داده‌ها
۶	۱-۱-۱ نمایش نمونه تصاویر
۶	۱-۱-۲ بررسی توزیع کلاس‌ها
۷	۱-۱-۳ پیش‌پردازش داده‌ها
۸	۲-۱ تقسیم دادگان
۸	۱-۲-۱ مروری بر روش Graph Sampling
۸	۲-۲-۱ روش تقسیم داده در این تمرین
۸	۳-۲-۱ تنظیم اندازه Batch
۹	۳-۱ آموزش مدل ResNet50
۹	۱-۳-۱ مروری بر ساختار مدل ResNet
۹	۲-۳-۱ تنظیم مدل برای مسئله بازشناسی
۹	۳-۳-۱ روند آموزش
۱۰	۴-۳-۱ نتایج آموزش
۱۰	۵-۳-۱ نمودار تغییرات خطا
۱۱	۴-۱ آموزش مدل BoTNet50
۱۱	۱-۴-۱ ایده اصلی معماری BoTNet
۱۱	۲-۴-۱ ساختار مدل پیاده‌سازی‌شده
۱۲	۳-۴-۱ تنظیمات آموزش
۱۲	۴-۴-۱ نتایج آموزش
۱۲	۵-۴-۱ نمودار تغییرات خطا و دقت

۱۳	۵-۱ بررسی نتایج BotNet
۱۳	۱-۵-۱ استخراج و نمایش نقشه توجه
۱۳	۲-۵-۱ نمونه نتایج
۱۵	۶-۱ تحلیل نتایج
۱۶	۲ مدل‌های زبانی بزرگ دیفیوژنی
۱۶	۱-۲ سوال‌های تئوری
۱۶	۱-۱-۲ تفاوت autoregressive و masked iterative generation
۱۷	۲-۱-۲ forward masking به عنوان noise model
۱۷	۳-۱-۲ چرا loss باید reweight شود؟
۱۸	۲-۲ داده، پرامپت و ابزارهای ارزیابی
۱۸	۱-۲-۲ ساخت pipeline داده
۱۹	۲-۲-۲ طراحی prompt به صورت chat format
۱۹	۳-۲-۲ SQL normalization و معیارها
۲۱	۳-۲ بارگزاری مدل و آموزش
۲۱	۱-۳-۲ بارگزاری مدل و توضیح مفاهیم جدید
۲۱	۲-۳-۲ پیاده سازی forward masking
۲۲	۳-۳-۲ مرز prompt/answer و ماسک کردن answer
۲۳	۴-۳-۲ loss و training loop
۲۶	۴-۲ تولید متن و ارزیابی
۲۶	۱-۴-۲ بخش امتیازی

فهرست تصاویر

۱	نمونه‌ای از تصاویر پنج کلاس مختلف	۶
۲	توزیع تعداد تصاویر در هر کلاس	۷
۳	تغییرات خطای آموزش و آزمون مدل ResNet50 در طول آموزش	۱۰
۴	تغییرات خطا و دقت مدل BoTNet50 در طول آموزش	۱۳
۵	نمونه‌هایی از نقشه توجه مدل BoTNet50 بر روی تصاویر دادگان	۱۴

فهرست جداول

۱ آمار طول schema و question در مجموعه آموزش (train) بر حسب کاراکتر و کلمه . . ۱۸

۱ بازشناسی تصاویر با استفاده از ترنسفورمر

۱-۱ آماده‌سازی داده‌ها

۱-۱-۱ نمایش نمونه تصاویر

پس از دریافت دادگان، ابتدا ساختار پوشه‌ها بررسی شد. هر پوشه نماینده‌ی یک کلاس (شناسه‌ی یک حیوان) است و تصاویر مربوط به همان کلاس درون آن قرار دارند. برای درک شهودی از داده‌ها، از پنج کلاس به‌صورت تصادفی نمونه‌گیری شد و از هر کلاس یک تصویر به‌صورت تصادفی انتخاب و نمایش داده شد.

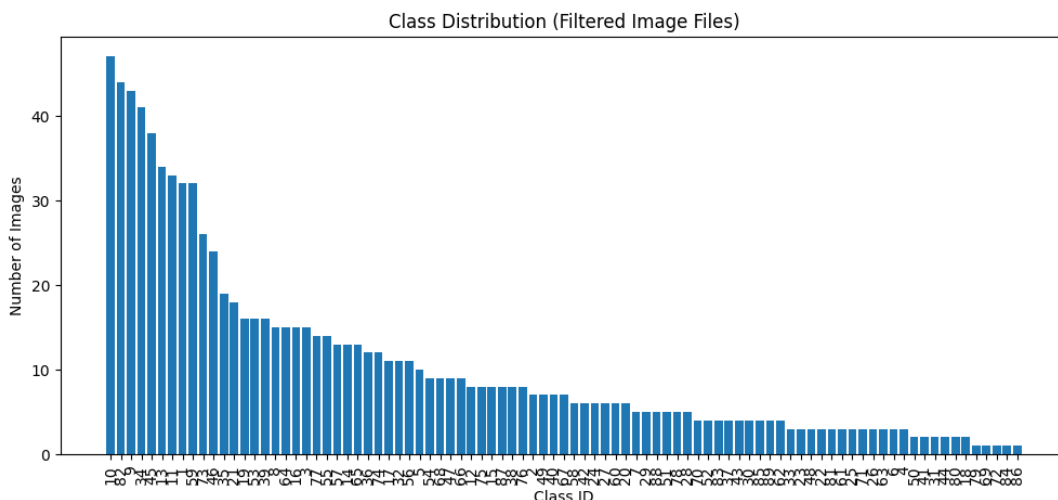


شکل ۱: نمونه‌ای از تصاویر پنج کلاس مختلف

این مرحله کمک می‌کند تا کیفیت تصاویر، شرایط نوری، زاویه دید، پس‌زمینه و میزان تنوع درون‌کلاسی بررسی شود. مشاهده اولیه نشان می‌دهد که تصاویر از نظر نور، زاویه و پس‌زمینه دارای تنوع هستند که می‌تواند فرآیند یادگیری مدل را تحت تأثیر قرار دهد.

۲-۱-۱ بررسی توزیع کلاس‌ها

در مرحله بعد، تعداد تصاویر موجود در هر کلاس محاسبه شد. هدف از این کار بررسی توازن یا عدم توازن داده‌ها است، زیرا عدم توازن کلاس‌ها می‌تواند منجر به بایاس مدل نسبت به کلاس‌های پرتکرار شود.



شکل ۲: توزیع تعداد تصاویر در هر کلاس

نمودار فوق نشان می‌دهد که توزیع داده‌ها در میان کلاس‌ها یکنواخت نیست و برخی کلاس‌ها تعداد تصاویر بسیار بیشتری نسبت به سایرین دارند. این موضوع می‌تواند باعث شود مدل در هنگام آموزش به سمت کلاس‌های پرتعداد متمایل شود.

۳-۱-۱ پیش‌پردازش داده‌ها

پس از تحلیل وضعیت داده‌ها، مراحل زیر برای پیش‌پردازش انجام شد:

- **نرمال‌سازی تصاویر:** مقادیر پیکسل‌ها به بازه استاندارد منتقل شدند تا فرآیند همگرایی در شبکه‌های عمیق بهبود یابد.
- **حذف کلاس‌های کم‌نمونه:** کلاس‌هایی که کمتر از ۵ تصویر داشتند حذف شدند. این کار به جلوگیری از بیش‌برازش (Overfitting) روی کلاس‌های بسیار کوچک کمک می‌کند.
- **افزایش داده (Data Augmentation):** به‌منظور افزایش تنوع داده و بهبود تعمیم‌پذیری مدل، از تبدیلاتی مانند چرخش تصادفی، برش تصادفی، وارونگی افقی و تغییرات جزئی روشنایی استفاده شد.

۲-۱ تقسیم دادگان

۱-۲-۱ مروری بر روش Graph Sampling

در مقاله مرجع، برای تقسیم دادگان از روش Graph Sampling استفاده شده است. در این روش، نمونه‌ها به‌صورت گره‌هایی در یک گراف در نظر گرفته می‌شوند و ارتباط میان آن‌ها (شباهت ویژگی‌ها یا تعلق به یک شناسه) به‌صورت یال مدل می‌شود. سپس با نمونه‌برداری ساختاریافته از این گراف، داده‌ها به بخش‌های آموزش و آزمون تقسیم می‌شوند.

مزیت این روش در مسائل Re-Identification آن است که وابستگی‌های درون‌کلاسی و بین‌کلاسی حفظ می‌شود و از نشت اطلاعات (Data Leakage) میان بخش آموزش و آزمون جلوگیری می‌شود. در مسائل بازشناسایی، اگر تصاویر بسیار مشابه از یک شناسه هم‌زمان در آموزش و آزمون قرار گیرند، ارزیابی مدل به‌صورت غیرواقعی خوش‌بینانه خواهد شد. استفاده از Graph Sampling می‌تواند این مشکل را کاهش دهد و ارزیابی دقیق‌تری ارائه دهد.

۲-۲-۱ روش تقسیم داده در این تمرین

در این تمرین، تقسیم داده‌ها به‌صورت معمول و بدون استفاده از Graph Sampling انجام شد. دادگان به نسبت ۸۰٪ برای آموزش و ۲۰٪ برای آزمون تقسیم گردید. از آنجا که هر کلاس نماینده‌ی یک شناسه مجزا است، برای جلوگیری از عدم توازن شدید، تقسیم داده به‌صورت درون‌کلاسی انجام شد؛ بدین معنا که برای هر کلاس، تصاویر آن کلاس به نسبت ۸۰ به ۲۰ بین آموزش و آزمون توزیع شدند. این کار باعث می‌شود که تمامی کلاس‌ها در هر دو بخش حضور داشته باشند.

۳-۲-۱ تنظیم اندازه Batch

اندازه Batch با توجه به محدودیت سخت‌افزاری انتخاب می‌شود. در این تمرین، مقدار Batch Size برابر با ۳۲ در نظر گرفته شد که تعادل مناسبی میان GPU و پایداری گرادینت‌ها ایجاد می‌کند.

۳-۱ آموزش مدل ResNet50

۱-۳-۱ مروری بر ساختار مدل ResNet

مدل Residual Network (ResNet) یکی از معماری‌های عمیق و پرکاربرد در حوزه بینایی ماشین است که با معرفی مفهوم Residual Learning مشکل محو شدن گرادیان (Vanishing Gradient) را در شبکه‌های عمیق کاهش داد.

ایده اصلی این معماری استفاده از اتصال‌های میان‌پُر (Skip Connections) است. به جای آن‌که هر بلوک مستقیماً نگاشت ورودی به خروجی را یاد بگیرد، اختلاف یا باقی‌مانده (Residual) بین آن‌ها را مدل می‌کند. این ساختار باعث می‌شود آموزش شبکه‌های بسیار عمیق پایدارتر و سریع‌تر انجام شود.

مدل ResNet50 شامل ۵۰ لایه عمیق است که از بلوک‌های Bottleneck Residual Block تشکیل شده و در مسائل طبقه‌بندی تصویر، بازشناسی (Re-Identification)، تشخیص شیء و استخراج ویژگی کاربرد گسترده‌ای دارد.

۲-۳-۱ تنظیم مدل برای مسئله بازشناسی

از نسخه ۵۰ لایه‌ای این معماری یعنی ResNet50 استفاده شد. از آنجا که مسئله حاضر یک مسئله طبقه‌بندی چندکلاسه است، لایه نهایی مدل متناسب با تعداد کلاس‌های دادگان تنظیم شد. برای تابع خطا از Cross Entropy Loss استفاده شد که انتخاب استاندارد برای مسائل طبقه‌بندی چندکلاسه محسوب می‌شود. بهینه‌سازی پارامترها نیز با استفاده از الگوریتم Adam Optimizer انجام شد که به دلیل نرخ یادگیری تطبیقی، همگرایی مناسبی در مراحل اولیه آموزش فراهم می‌کند.

۳-۳-۱ روند آموزش

مدل به مدت حداقل ۲۰ دوره آموزشی (Epoch) آموزش داده شد (در این آزمایش ۳۰ دوره اجرا شد). در هر دوره، مقدار خطا و دقت برای هر دو بخش آموزش و آزمون محاسبه گردید تا روند یادگیری مدل بررسی شود.

۴-۳-۱ نتایج آموزش

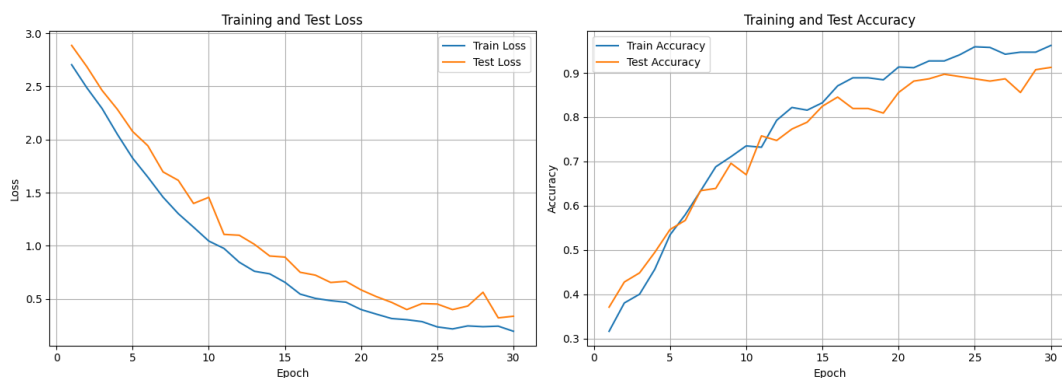
در انتهای فرآیند آموزش، نتایج زیر حاصل شد:

- دقت آموزش: ۹۶,۱۹٪
- دقت آزمون: ۹۱,۲۴٪
- خطای آموزش: ۰,۱۹۵۵
- خطای آزمون: ۰,۳۳۷۰

مشاهده می‌شود که مدل به همگرایی مناسبی رسیده است. اختلاف میان دقت آموزش و آزمون نسبتاً محدود است که نشان می‌دهد مدل دارای تعمیم‌پذیری قابل قبول بوده و دچار بیش‌برازش شدید نشده است.

۵-۳-۱ نمودار تغییرات خطا

در شکل زیر، روند تغییرات خطای آموزش و آزمون بر حسب تعداد دوره‌های آموزشی نمایش داده شده است.



شکل ۳: تغییرات خطای آموزش و آزمون مدل ResNet50 در طول آموزش

کاهش تدریجی خطای آموزش و پایدار شدن خطای آزمون نشان‌دهنده یادگیری مؤثر مدل است. در دوره‌های پایانی، فاصله اندکی میان دو منحنی مشاهده می‌شود که بیانگر وجود مقدار محدودی بیش‌برازش طبیعی در شبکه‌های عمیق است.

۴-۱ آموزش مدل BoTNet50

۱-۴-۱ ایده اصلی معماری BoTNet

مدل BoTNet (Bottleneck Transformer) ترکیبی از ساختار کانولوشنی CNN و مکانیزم توجه Self-Attention است. در این معماری، به جای استفاده کامل از کانولوشن در لایه‌های عمیق، بخشی از بلوک‌های انتهایی شبکه با ماژول توجه چندسری (Multi-Head Self Attention) جایگزین می‌شوند. ایده اصلی این است که کانولوشن‌ها در استخراج ویژگی‌های محلی بسیار قدرتمند هستند، اما در مدل‌سازی وابستگی‌های بلندبرد (Long-Range Dependencies) محدودیت دارند. در مقابل، مکانیزم توجه قادر است ارتباط بین تمامی نواحی تصویر را به صورت سراسری مدل کند. بنابراین BoTNet50 تلاش می‌کند مزایای هر دو ساختار CNN و Transformer را به صورت همزمان بهربرداری کند.

۲-۴-۱ ساختار مدل پیاده‌سازی شده

در این تمرین، معماری ResNet50 به عنوان Backbone استفاده شد و پس از مرحله چهارم شبکه، یک بلوک BotBlock اضافه گردید. این بلوک شامل مراحل زیر است:

- کاهش ابعاد کانال‌ها با کانولوشن 1×1

- اعمال مکانیزم Multi-Head Self Attention

- بازگرداندن ابعاد کانال‌ها

- اتصال میان‌بر (Residual Connection)

وجود اتصال باقیمانده باعث حفظ پایداری آموزش مشابه ResNet می‌شود، در حالی که لایه توجه امکان مدل‌سازی روابط سراسری بین نواحی تصویر را فراهم می‌کند. همچنین خروجی لایه توجه ذخیره شد تا در صورت نیاز در تحلیل‌های بعدی قابل استفاده باشد.

۳-۴-۱ تنظیمات آموزش

فرآیند آموزش مشابه مدل قبلی انجام شد:

- تابع خطا: Cross Entropy Loss

- بهینه‌ساز: Adam

- تعداد دوره‌های آموزشی: ۳۰ Epoch

- اندازه Batch: ۳۲

۴-۴-۱ نتایج آموزش

در انتهای آموزش، نتایج زیر به دست آمد:

- دقت آموزش: ۱۰۰٪

- دقت آزمون: ۹۶,۹۱٪

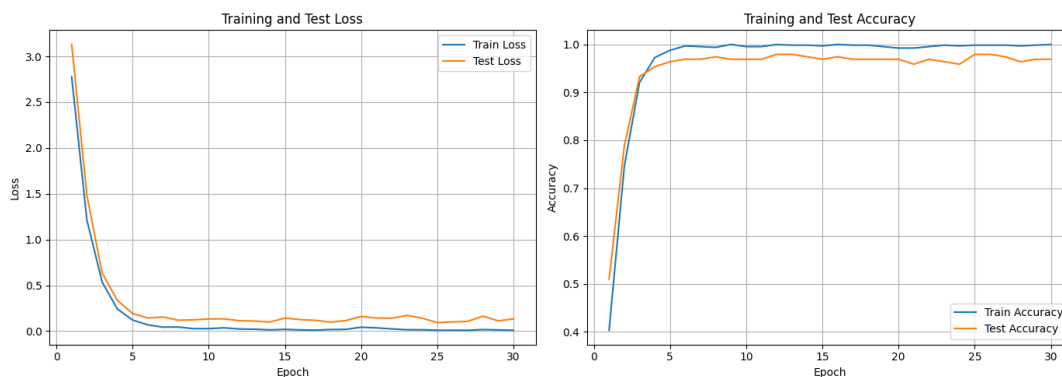
- خطای آموزش: ۰,۰۰۹۴

- خطای آزمون: ۰,۱۳۴۴

کاهش قابل توجه خطای آموزش و افزایش دقت آزمون نسبت به مقدار اولیه نشان‌دهنده همگرایی مناسب مدل است. با توجه به اینکه دقت آموزش به مقدار بسیار بالا رسیده است، مدل ظرفیت بالایی در یادگیری داده‌ها دارد.

۵-۴-۱ نمودار تغییرات خطا و دقت

در شکل زیر، روند تغییرات خطا و دقت در بخش آموزش و آزمون نمایش داده شده است.



شکل ۴: تغییرات خطا و دقت مدل BoTNet50 در طول آموزش

همان‌طور که مشاهده می‌شود، منحنی خطای آموزش به‌سرعت کاهش یافته و به مقدار بسیار پایینی رسیده است. خطای آزمون نیز روند کاهشی داشته و در انتهای آموزش پایدار شده است که نشان‌دهنده یادگیری مؤثر ویژگی‌های سراسری تصویر توسط مکانیزم توجه است.

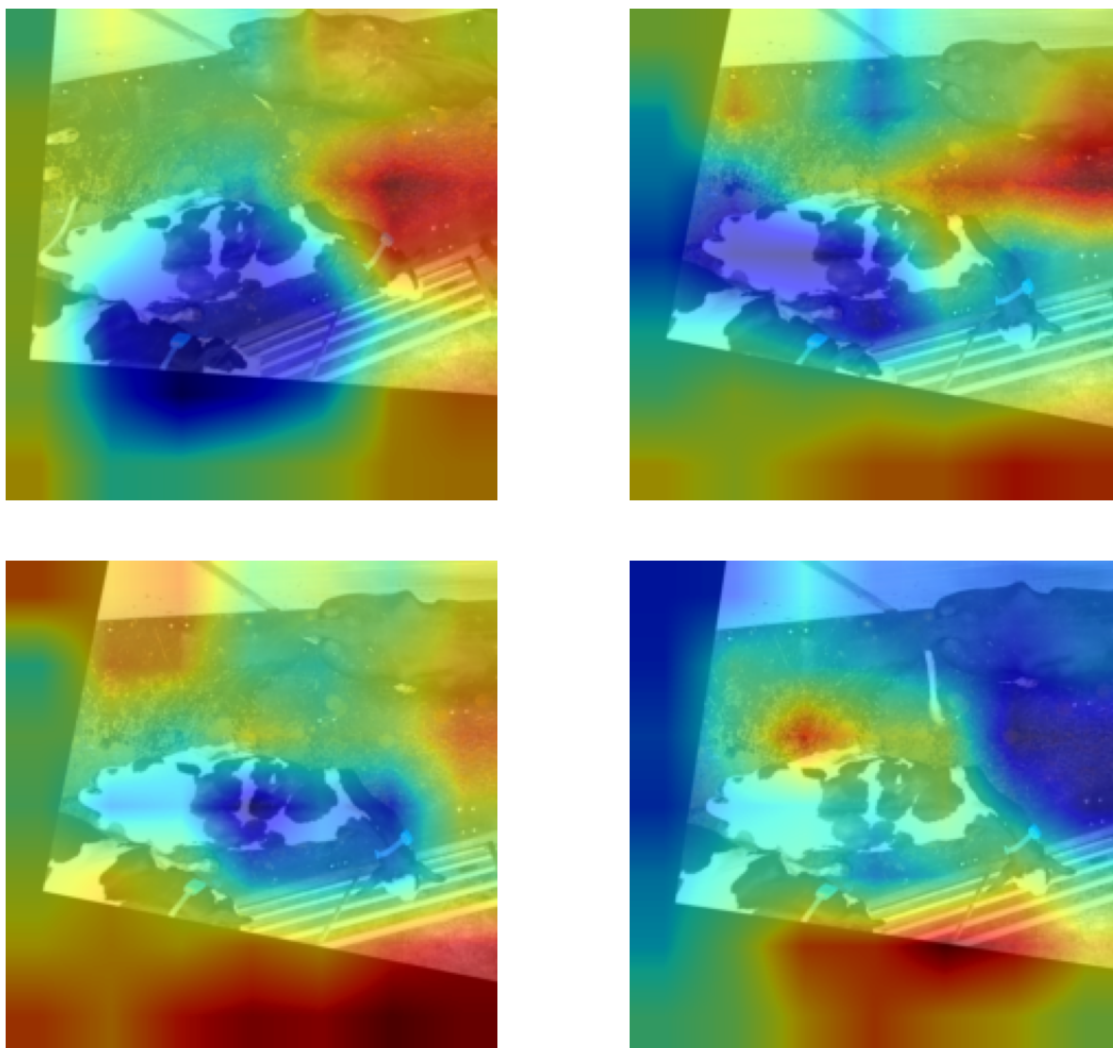
۵-۱ بررسی نتایج BotNet

۱-۵-۱ استخراج و نمایش نقشه توجه

به‌منظور تحلیل رفتار مدل BoTNet50، خروجی لایه توجه (Attention Layer) استخراج و به‌صورت Heatmap بر روی تصویر اصلی نمایش داده شد. برای این کار، تصویر ابتدا به ابعاد ورودی شبکه تغییر اندازه داده شد و نرمال‌سازی انجام گرفت. سپس خروجی لایه توجه استخراج و متناسب با ابعاد تصویر Resize شد. در نهایت، تصویر از حالت نرمال خارج شده و نقشه توجه به‌صورت همپوشان روی آن نمایش داده شد.

۲-۵-۱ نمونه نتایج

در شکل زیر، چهار نمونه از تصاویر به همراه نقشه توجه مدل نمایش داده شده‌اند.



شکل ۵: نمونه‌هایی از نقشه توجه مدل BoTNet50 بر روی تصاویر دایگان

همانطور که مشخص است مدل بیشتر روی ویژگی‌های محیطی متمرکز شده است تا پوست گاو(تصاویر سمت چپ به خوبی این موضوع را نشان می‌دهند) همچنین تصویرهای سمت راست می‌توان گفت نمونه خوبی برای این قضیه هستند که مدل با استفاده از اطلاعات نامربوط تصاویر را طبقه‌بندی کرده است.

۶-۱ تحلیل نتایج

در این آزمایش، عملکرد دو مدل ResNet50 (مدل کانولوشنی) و BoTNet50 (مدل مبتنی بر Transformer) مورد مقایسه قرار گرفت. بر اساس نتایج به دست آمده، مدل BoTNet50 عملکرد بهتری از نظر دقت نهایی و پایداری در فرآیند آموزش نشان داد. استفاده از مکانیزم Self-Attention در BoTNet باعث می شود مدل بتواند وابستگی های بلندبرد و الگوهای کلی تری را استخراج کند، در حالی که ResNet50 بیشتر بر ویژگی های محلی متمرکز است.

با توجه به دانش قبلی انتظار می رفت مدل ترنسفورمری در مسئله ای مانند Re-Identification که وابسته به الگوهای بافتی سراسری است عملکرد بهتری داشته باشد، که این موضوع در نتایج مشاهده شد.

با این حال، مدلی که عملکرد ضعیف تری داشت (ResNet50) در شرایطی مانند افزایش حجم داده های آموزشی، استفاده از Data Augmentation گسترده تر و انجام Hyperparameter Tuning دقیق تر می توانست عملکرد رقابتی تری از خود نشان دهد، زیرا شبکه های کانولوشنی معمولاً در صورت دسترسی به داده ی کافی بسیار قدرتمند عمل می کنند.

در صورتی که داده ها نامتوازن باشند، معمولاً کلاس هایی که تعداد نمونه کمتری دارند با کاهش دقت مواجه می شوند. در این آزمایش نیز مشاهده شد که برای برخی کلاس های کم نمونه، افت دقت در هر دو مدل وجود دارد، اما این افت در ResNet50 محسوس تر بود. این موضوع نشان می دهد که مدل مبتنی بر توجه (Attention-Based) توانایی بهتری در تعمیم به کلاس های کم داده داشته است.

در مجموع، اختلاف دقت بین خروجی دو مدل معنادار بوده و می توان نتیجه گرفت که برای این نوع مسئله، بهره گیری از ساختارهای مبتنی بر توجه مانند BoTNet50 انتخاب مناسب تری نسبت به معماری صرفاً کانولوشنی مانند ResNet50 است.

۲ مدل‌های زبانی بزرگ دیفیوژنی

۱-۲ سوال‌های تنوری

۱-۱-۲ تفاوت autoregressive و masked iterative generation

مدل‌های autoregressive متن را به صورت ترتیبی و از چپ به راست تولید می‌کنند؛ به این معنا که احتمال هر توکن جدید را با شرط بر تمام توکن‌های قبلی محاسبه می‌کنند. در مرحله آموزش، معمولاً از روش teacher forcing استفاده می‌شود؛ یعنی در هر گام، توکن صحیح قبلی به مدل داده می‌شود تا توکن بعدی را پیش‌بینی کند و تابع زیان بر اساس اختلاف بین توکن پیش‌بینی‌شده و توکن مرجع محاسبه می‌گردد. در مرحله تولید، مدل با یک پیشوند (prompt) آغاز کرده و به صورت گام به گام توکن‌های بعدی را نمونه برداری یا انتخاب می‌کند تا به انتهای پاسخ برسد. در مقابل، مدل‌های masked-prediction در آموزش بخشی از توکن‌های ورودی را به صورت تصادفی با یک نشانه ماسک جایگزین می‌کنند و مدل می‌آموزد توکن‌های پنهان شده را با استفاده از کل زمینه (چپ و راست) بازسازی کند. در حالت تولید، به ویژه در روش masked iterative generation، ابتدا یک توالی ناقص یا کاملاً ماسک شده ساخته می‌شود و سپس در چندین مرحله، مدل بخشی از توکن‌های نامطمئن را پیش‌بینی و به روز رسانی می‌کند تا به توالی نهایی برسد.

در مسئله Text-to-SQL یکی از مزایای روش masked iterative generation این است که امکان بازنگری تصمیم‌های قبلی را فراهم می‌کند؛ برای مثال اگر در ابتدای تولید، نام جدول اشتباهی انتخاب شود، در تکرارهای بعدی می‌توان آن را اصلاح کرد، زیرا مدل الزاماً به یک مسیر خطی و غیرقابل بازگشت محدود نیست. اما یک محدودیت آن می‌تواند ناپایداری یا همگرایی کند باشد، زیرا ممکن است چندین بار نیاز به بازنویسی بخش‌های مختلف پرس و جو وجود داشته باشد. در مقابل، در مدل autoregressive اگر در ابتدای تولید یک تصمیم اشتباه مانند انتخاب نادرست نام ستون رخ دهد، این خطا در توکن‌های بعدی منتشر شده و معمولاً تا انتهای دنباله باقی می‌ماند، زیرا تولید به صورت یک طرفه و بدون بازگشت انجام می‌شود. بنابراین روش autoregressive ساده‌تر و پایدارتر در مسیر تولید است، اما در برابر خطاهای اولیه انعطاف پذیری کمتری دارد، در حالی که روش masked iterative generation انعطاف پذیرتر ولی پیچیده‌تر از نظر فرایند تولید است.

۲-۱-۲ forward masking به عنوان noise model

فرض کنید یک توالی تمیز از توکن‌ها در اختیار داریم. در روش forward masking، برای هر نمونه یک احتمال ماسک کردن P انتخاب می‌شود و به ازای هر توکن، با احتمال P آن توکن با [MASK] جایگزین می‌شود تا نسخه نویزی توالی ساخته شود. به این ترتیب، مدل در هنگام آموزش یاد می‌گیرد که بخش‌هایی از توالی را که گم شده یا مخدوش شده بازسازی کند و به پیش‌بینی توکن‌های اصلی توانمند شود.

اینکه مقدار P در آموزش ثابت نباشد و گاهی کم و گاهی زیاد باشد مفید است زیرا مدل را قادر می‌سازد نسبت به سطوح مختلف نویز مقاوم شود و بتواند در موقعیت‌های با خطای کم یا زیاد عملکرد مناسبی داشته باشد. اگر همیشه یک مقدار ثابت، مانند ۱۵٪، استفاده شود، ممکن است مدل فقط به همان سطح نویز عادت کند و در حالت‌هایی که نویز بیشتر یا کمتر است دچار خطا شود. علاوه بر این، ثابت بودن P می‌تواند منجر به همگرایی ضعیف‌تر و کاهش انعطاف‌پذیری مدل شود، زیرا مدل تنها با یک نوع الگوی مخدوش شدن توکن‌ها مواجه شده و برای شرایط متفاوت آماده نیست.

۳-۱-۲ چرا loss باید reweight شود؟

در این تمرین، برای توکن‌های شده ماسک loss با یک ضریب مربوط به احتمال ماسک کردن (P) reweight می‌شود. دلیل این کار این است که اگر این بازوزن دهی انجام نشود، نمونه‌هایی که تعداد کمی توکن ماسک شده دارند، سهم بسیار کمی در محاسبه loss خواهند داشت و در نتیجه اثر آنها در یادگیری مدل ناچیز می‌شود. به عبارت دیگر، مدل در مواجهه با نمونه‌هایی که بخش زیادی از توکن‌ها دست نخورده باقی مانده‌اند، به خوبی یاد نمی‌گیرد چگونه بخش‌های مخدوش یا گم شده را بازسازی کند.

هدف از reweight کردن loss این است که همه نمونه‌ها، صرف‌نظر از تعداد توکن‌های ماسک شده، تأثیر متعادلی بر یادگیری داشته باشند. این تعادل باعث می‌شود که مدل بتواند نسبت به سطوح مختلف نویز مقاوم باشد و یادگیری آن بر اساس تعداد توکن‌های ماسک شده، منحرف یا ناپایدار نشود. در واقع، reweighting تضمین می‌کند که نمونه‌های کم نویز و پر نویز هر دو سهم مناسبی در بهبود وزن‌های مدل دارند و یادگیری به صورت یکنواخت و پایدار انجام شود.

۲-۲ داده، پرامپت و ابزارهای ارزیابی

۱-۲-۲ ساخت pipeline داده

در این پروژه، مجموعه داده‌ها به سه بخش تقسیم شده‌اند: آموزش (train)، ارزیابی (valid) و آزمایش (test) که به ترتیب شامل ۱۰۰۰، ۳۰۰ و ۳۰۰ نمونه هستند.

طول schema و question در مجموعه آموزش به دو صورت بر حسب کاراکتر و بر حسب کلمه مورد بررسی قرار گرفت. نتایج به شرح زیر است:

متن	میانگین	انحراف معیار	بیشینه
Schema (Character Count)	272.79	138.68	1041
Question (Character Count)	83.9	26.75	197
Schema (Word Count)	32.44	17.53	149
Question (Word Count)	13.99	4.27	32

جدول ۱: آمار طول schema و question در مجموعه آموزش (train) بر حسب کاراکتر و کلمه

به طور معمول، طول schema بسیار بیشتر از question است و بنابراین باید روی آن تمرکز کرد. برای کنترل طول schema، از یک روش ساده استفاده شد: ابتدا اطلاعات اضافی از schema حذف شد و تنها نام جدول‌ها و ستون‌ها نگه داشته شد تا متن اصلی کوتاه شود. سپس در زمان توکن‌سازی، MAX_LEN=512 تنظیم شد و truncation=True فعال گردید تا اگر طول prompt بیش از حد بزرگ باشد، توکنایزر به صورت خودکار متن را کوتاه کند. این اطمینان داده شد که بخش پاسخ SQL حذف نشود و همواره فضای کافی برای آن وجود داشته باشد. این تصمیم باعث شد که طول طولانی schema مدیریت شود و از حذف بخش‌های مهم پاسخ جلوگیری گردد.

یک تابع ساده‌سازی schema طراحی شد که متن schema را پردازش می‌کند و تنها نام جدول‌ها و ستون‌ها را نگه می‌دارد و اطلاعات اضافی را حذف می‌کند. این روش باعث شد طول schema کاهش یابد و توکنایزر بتواند بدون از دست رفتن پاسخ SQL متن را به طول مناسب برای مدل محدود کند. به بیان ساده، با توجه به اینکه طول schema معمولاً بسیار بیشتر از question است، تمرکز اصلی روی کوتاه کردن آن بود تا فضای کافی برای پاسخ حفظ شود.

۲-۲-۲ طراحی prompt به صورت chat format

قالب پیام برای مدل LLaDA به صورت chat-format طراحی شد و شامل سه نقش اصلی است:

- System message: این پیام مدل را مجبور می‌کند که SQL تولید کند و هیچ توضیح یا کامنت اضافه ندهد.:

- User message: شامل schema ساده شده و پرسش (question) است.

- Assistant message: در مرحله آموزش، gold SQL در این پیام قرار می‌گیرد. در نسخه بدون جواب، این پیام خالی است تا مرز شروع تولید پاسخ مشخص شود.

با طراحی این قالب، پاسخ SQL تنها در پیام assistant قرار می‌گیرد. این کار باعث می‌شود که در طول آموزش و ماسکینگ، تنها بخش پاسخ نویزی (answer) نویزی شود و بخش prompt (system + user) دست‌نخورده باقی بماند. بنابراین مرز بین prompt و answer به طور دقیق مشخص است و مدل نمی‌تواند بخش schema یا پرسش را تغییر دهد.

یک نسخه از prompt بدون gold SQL نیز ایجاد شد که شامل system، user و هدر assistant اما بدون محتوا است. این نسخه برای شناسایی دقیق نقطه شروع پاسخ تولید شده توسط مدل استفاده می‌شود و به ویژه در ارزیابی exact match اهمیت دارد. به دلیل اینکه مدل تنها در بخش assistant باید SQL تولید کند، اطمینان حاصل می‌شود که محاسبه exact match با مقایسه مستقیم پاسخ تولید شده با gold SQL دقیق انجام شود. وجود پیام system و دستور صریح "output only SQL" برای ارزیابی exact match ضروری است. بدون این پیام، مدل ممکن است توضیحات اضافی یا کامنت تولید کند که با gold SQL مطابقت ندارد و باعث کاهش معیار exact match می‌شود. بنابراین message system تضمین می‌کند که خروجی مدل فقط SQL باشد و ارزیابی معیار دقیق و معتبر باشد.

۳-۲-۲ SQL normalization و معیارها

برای استفاده در معیار normalized exact match، یک تابع نرمال‌سازی SQL طراحی شد که مراحل زیر را اعمال می‌کند:

- تبدیل تمام حروف به lowercase

- حذف کاراکترهای نقل قول (” و ’) و بکتیک (‘)

- حذف سمی‌کالن پایانی (;)

- یکسان‌سازی فاصله‌ها (چند فاصله متوالی به یک فاصله تبدیل شود)

- حذف فاصله‌های اضافی ابتدای و انتهای رشته

این عملیات باعث می‌شود تا مقایسه متن SQL بین پیش‌بینی مدل و پاسخ gold پایدارتر و کمتر به تفاوت‌های فرمی حساس باشد.

دو تابع ارزیابی تعریف شد:

- **Raw Exact Match**: مقایسه مستقیم رشته پیش‌بینی و مرجع. خروجی True یا False است.

- **Normalized Exact Match**: ابتدا هر دو رشته با تابع normalize_sql نرمال‌سازی می‌شوند و

سپس مقایسه دقیق انجام می‌شود. این روش باعث می‌شود تفاوت‌های فرمی مانند حروف بزرگ/کوچک، فاصله‌های اضافی یا سمی‌کالن‌های پایانی، در ارزیابی تأثیر نداشته باشند.

یکی از محدودیت‌های این معیارها این است که دو query ممکن است از نظر معنایی برابر باشند اما از نظر متن متفاوت. برای مثال:

- `SELECT name FROM students WHERE age > 18`

- `SELECT name FROM students WHERE 18 < age`

این دو query از نظر نتیجه اجرا برابر هستند، اما هر دو تابع raw و normalized exact match آن‌ها را متفاوت ارزیابی می‌کنند زیرا متن دقیق آن‌ها مشابه نیست. این محدودیت باعث می‌شود که معیار exact match همیشه قادر به تشخیص برابری معنایی نباشد.

۳-۲ بارگذاری مدل و آموزش

۱-۳-۲ بارگذاری مدل و توضیح مفاهیم جدید

4-bit quantization به معنی کاهش دقت پارامترهای مدل از float32 یا float16 به تنها ۴ بیت است. این کاهش دقت باعث می‌شود حجم حافظه‌ای که برای ذخیره وزن‌ها و بارگذاری مدل مورد نیاز است به شدت کاهش یابد و امکان استفاده از مدل‌های بزرگ روی GPU با حافظه محدود فراهم شود. با وجود کاهش دقت، معمولاً دقت نهایی مدل در تولید پاسخ‌ها به میزان قابل قبولی حفظ می‌شود.

LoRA یا Low-Rank Adaptation یک روش بهینه‌سازی است که به جای آموزش همه پارامترهای مدل، فقط چند ماتریس کم‌رتبه اضافه را آموزش می‌دهد. مزیت اصلی این روش این است که تعداد پارامترهای قابل آموزش بسیار کمتر می‌شود و حافظه مورد نیاز برای گرادیان‌ها کاهش می‌یابد. همچنین، مدل پایه ثابت باقی می‌ماند و تنها این پارامترهای جدید یاد می‌گیرند که چگونه مدل را برای وظیفه خاصی مانند تولید SQL تنظیم کنند.

KV cache یک حافظه داخلی برای ذخیره کلیدها و مقادیر (keys و values) در self-attention است تا در نسل توالی‌ها به صورت autoregressive، هر مرحله نیاز به محاسبه دوباره تمام توکن‌های قبلی نباشد. در سبک ما که ماسکینگ و آموزش روی کل توکن‌ها انجام می‌شود، از آنجایی که پاسخ‌نویزی تنها در بخش assistant صورت می‌گیرد و نه به صورت تک‌تک توکن‌های autoregressive، KV cache کاربرد ندارد و نیازی به فعال‌سازی آن در این حالت نیست.

۲-۳-۲ پیاده‌سازی forward masking

در پیاده‌سازی ارائه‌شده:

- طول بخش پاسخ برای هر نمونه با کم کردن `prompt_length` از طول کل توکن‌ها محاسبه می‌شود.
- مقدار `p_mask` برای هر نمونه از توزیع یکنواخت $\mathcal{U}(0, 1)$ نمونه‌گیری می‌شود.
- تعداد توکن‌های ماسک‌شونده برابر است با:

$$\lceil p_mask \times \text{answer_length} \rceil$$

به طوری که اگر طول پاسخ بزرگتر از صفر باشد، حداقل یک توکن ماسک می‌شود.

- موقعیت‌های ماسک‌شونده به صورت تصادفی و بدون تکرار از ناحیه پاسخ انتخاب می‌شوند.

وجود masked_indices باعث می‌شود loss تنها روی موقعیت‌های ماسک‌شده محاسبه شود. همچنین استفاده از p_mask امکان وزن‌دهی تطبیقی را فراهم می‌کند تا نمونه‌هایی که تعداد کمی توکن ماسک دارند، نسبت به نمونه‌هایی که تقریباً کامل ماسک شده‌اند، تأثیر نامتوازن در آموزش نداشته باشند. به این ترتیب، مدل به جای حفظ کردن کل پاسخ، مجبور می‌شود ساختار نحوی و معنایی پاسخ را درک کرده و بخش‌های حذف‌شده را بازسازی کند، که این امر به بهبود تعمیم‌پذیری مدل کمک می‌کند.

۲-۳-۳ مرز prompt/answer و ماسک کردن answer

در مرحله آموزش، تابع forward_mask_answer_only ممکن است به صورت اولیه ماسک را روی کل دنباله اعمال کند. اما با داشتن prompt_length می‌توان به سادگی ناحیه prompt را بازگردانی (restore) کرد تا:

- توکن‌های مربوط به prompt همیشه تمیز باقی بمانند.

- فقط ناحیه پاسخ (answer region) نویزی یا ماسک شود.

در نتیجه، مدل همواره ورودی شرطی صحیح را دریافت می‌کند و تنها وظیفه بازسازی بخش پاسخ را بر عهده دارد.

نویز کردن prompt از چند جهت نادرست است:

۱. **برهم زدن شرط مسئله:** prompt همان اطلاعات شرطی (سؤال و اسکیمای پایگاه داده) است. اگر این بخش نویزی شود، مدل دیگر مسئله واقعی را دریافت نمی‌کند و یادگیری به صورت ناپایدار انجام می‌شود.
۲. **تغییر هدف یادگیری:** هدف ما آموزش مدل برای بازسازی پاسخ SQL است، نه بازسازی سؤال یا اسکیمای ورودی. اگر prompt نیز ماسک شود، مسئله به نوعی denoising autoencoder عمومی تبدیل می‌شود و تمرکز از تولید پاسخ صحیح برداشته می‌شود.

۳. **ناهمخوانی با سناریوی تست:** در زمان ارزیابی، prompt همیشه تمیز و کامل به مدل داده می‌شود. بنابراین اگر در آموزش prompt نویزی شود، توزیع داده آموزش و تست با یکدیگر همخوان نخواهند بود.

۴. **محاسبه نادرست loss:** در این طراحی، loss فقط روی masked_indices محاسبه می‌شود. اگر prompt نیز ماسک شود، مدل مجبور خواهد شد بخش شرطی را نیز پیش‌بینی کند، در حالی که این بخش باید ثابت و داده‌شده فرض شود.

بنابراین، دو بار توکنایز کردن هر نمونه این امکان را فراهم می‌کند که:

- مرز دقیق بین prompt و پاسخ مشخص شود،
- فرآیند نویزگذاری فقط روی ناحیه پاسخ اعمال گردد،
- هدف یادگیری به صورت دقیق بر بازسازی پاسخ متمرکز بماند.

این طراحی باعث پایداری بیشتر آموزش و همخوانی کامل با سناریوی تولید autoregressive در زمان ارزیابی خواهد شد.

۴-۳-۲ loss و training loop

در این بخش، من تابع محاسبه‌ی loss را به گونه‌ای طراحی کرده‌ام که مدل فقط روی بخش پاسخ (یعنی SQL) آموزش ببیند و قسمت prompt همواره تمیز و بدون نویز باقی بماند. در ادامه توضیح داده می‌شود که در هر قسمت چه تصمیمی گرفته شده و دلیل آن چه بوده است.

(۱) تبدیل ورودی‌ها به Tensor و انتقال به Device

در ابتدای تابع، مقادیر input_ids، attention_mask و prompt_length به torch.Tensor تبدیل شده و به device (مثلاً GPU) منتقل می‌شوند. این کار باعث می‌شود تمام محاسبات بعدی روی همان سخت‌افزار انجام شود و از خطاهای ناسازگاری بین CPU و GPU جلوگیری گردد. همچنین اطمینان حاصل می‌شود که نوع داده‌ها برای محاسبات گرایانی مناسب است.

(۲) ساخت نسخه‌ی نویزی فقط برای پاسخ

در مرحله‌ی بعد، با استفاده از تابع `make_noisy_batch` یک نسخه‌ی نویزی از توکن‌ها ساخته می‌شود. نکته‌ی مهم این است که ماسک‌گذاری فقط در ناحیه‌ی پاسخ انجام می‌شود و طول `prompt` از طریق `prompt_length` مشخص شده است.

این تصمیم به این دلیل گرفته شد که `prompt` نقش شرط اولیه‌ی مسئله را دارد و نباید تخریب شود. اگر `prompt` نیز ماسک شود، مدل مجبور می‌شود هم سؤال را بازسازی کند و هم پاسخ را، که باعث تداخل هدف آموزشی می‌شود. هدف اصلی، یادگیری تولید SQL صحیح بر اساس یک ورودی مشخص است، نه بازسازی کل دنباله.

(۳) اجرای مدل روی ورودی نویزی

مدل با ورودی نویزی اجرا می‌شود و خروجی آن شامل logits با ابعاد $B \times T \times V$ است (اندازه‌ی بچ، طول دنباله و اندازه‌ی واژگان). این مرحله در واقع همان پیش‌بینی توکن‌های اصلی بر اساس نسخه‌ی ماسک‌شده است.

(۴) تخت کردن ابعاد برای محاسبه‌ی Cross-Entropy

برای محاسبه‌ی cross-entropy، ابعاد logits و اهداف به شکل دویبعی تبدیل می‌شوند. این کار باعث می‌شود بتوان loss را به صورت توکن‌به‌توکن محاسبه کرد. در این مرحله هنوز هیچ ماسکی اعمال نشده است و loss برای تمام موقعیت‌ها محاسبه می‌شود.

(۵) حذف تأثیر توکن‌های Padding

از آنجا که برخی دنباله‌ها کوتاه‌تر هستند و با pad token پر شده‌اند، یک pad mask تعریف می‌شود تا این موقعیت‌ها در محاسبه‌ی نهایی loss لحاظ نشوند. این کار از آن جهت مهم است که padding اطلاعات واقعی ندارد و نباید در به‌روزرسانی پارامترها نقش داشته باشد.

۶ اعمال ماسک نهایی فقط روی توکن‌های مخفی شده

یک ماسک نهایی ساخته می‌شود که فقط شامل موقعیت‌هایی است که هم ماسک شده‌اند و هم padding نیستند. به این ترتیب، loss فقط روی همان توکن‌هایی محاسبه می‌شود که مدل موظف به بازسازی آن‌ها بوده است. این کار باعث می‌شود مدل دقیقاً روی مسئله‌ی مورد نظر (پر کردن جای خالی در پاسخ) آموزش ببیند و از نویز اضافی جلوگیری شود.

۷ وزن‌دهی معکوس بر اساس نسبت ماسک

برای هر نمونه در بچ، یک مقدار تصادفی t به عنوان نسبت ماسک انتخاب شده است. اگر نمونه‌ای درصد زیادی از توکن‌هایش ماسک شود، به طور طبیعی loss بیشتری تولید می‌کند. برای جلوگیری از غلبه‌ی چنین نمونه‌هایی، از وزن‌دهی معکوس نسبت به t استفاده شده است (یعنی $1 / p_mask$). این کار باعث می‌شود نمونه‌هایی که توکن‌های بیشتری ماسک شده‌اند، به صورت نامتناسب بر گرادیان اثر نگذارند و تعادل بین نمونه‌های کم‌ماسک و پر‌ماسک برقرار شود. در نتیجه آموزش پایدارتر خواهد بود.

۸ نرمال‌سازی نهایی Loss

در انتها، مجموع loss وزن‌دهی‌شده بر مجموع وزن‌های مؤثر تقسیم می‌شود. این کار باعث می‌شود مقدار loss نسبت به تعداد توکن‌های ماسک‌شده مستقل شود و مقیاس آن بین بچ‌های مختلف قابل مقایسه باقی بماند.

جمع‌بندی طراحی

در مجموع، طراحی این تابع با سه هدف اصلی انجام شده است:

- تمیز نگه داشتن prompt و تمرکز آموزش فقط بر بخش پاسخ،
- محاسبه‌ی دقیق loss فقط روی توکن‌های واقعاً ماسک‌شده،
- متعادل‌سازی اثر نمونه‌های مختلف از نظر شدت ماسک‌گذاری.

این طراحی باعث می‌شود فرآیند آموزش پایدارتر، هدفمندتر و هم‌استا با هدف اصلی مدل یعنی تولید صحیح

SQL باشد.

۴-۲ تولید متن و ارزیابی

پیرامون این قسمت، من سعیم رو برای اینکه بتونم از کد ران بگیرم انجام دادم اما نتونستم. علاوه بر قطع و وصلی مداوم kaggle یک سری مشکلات مربوط به سازگاری لایبرری‌ها در این محیط مانع ران گرفتن میشد. زمان زیادی گذاشتم اما یا نمیشد ران گرفت یا در ازای ران گرفتن دقت بسیار پایینی دریافت میکردم. حین سعی و تلاش آخرین نسخه ای که تا قبل از ددلاین انجام داده بودم رو گذاشتم. بنابراین بخش ارزیابی نتایج انجام نشد. اما پیرامون این بخش تلاشم رو کردم اما نتونستم به نتیجه برسم:)

۱-۴-۲ بخش امتیازی

در این بخش پنج ایده‌ی خلاقانه و اصولی برای بهبود کیفیت، پایداری یا کارایی روش پیشنهادی مطرح می‌شود. هر ایده شامل توضیح تغییر پیشنهادی، دلیل مفید بودن آن و اثر مورد انتظار بر عملکرد مدل است. به جای آنکه احتمال ماسک شدن هر توکن صرفاً بر اساس یک مقدار تصادفی t تعیین شود، می‌توان یک مکانیزم adaptive masking طراحی کرد که اهمیت توکن‌ها را در نظر بگیرد. برای مثال، در کوئری‌های SQL برخی توکن‌ها مانند نام ستون‌ها، جدول‌ها، عملگرهای شرطی و کلمات کلیدی اصلی (مانند SELECT و WHERE) نقش حیاتی‌تری دارند. می‌توان با استفاده از موقعیت توکن در ساختار SQL یا حتی با بهرگیری از میزان توجه (attention score) مدل در یک پیش‌اجرای اولیه، وزن اهمیت هر توکن را تخمین زد و توکن‌های کلیدی‌تر را با احتمال بیشتری ماسک کرد. انتظار می‌رود این کار باعث شود مدل تمرکز بیشتری بر بخش‌های ساختاری و حساس داشته باشد، در نتیجه دقت نهایی افزایش یابد، در حالی که هزینه‌ی محاسباتی تقریباً ثابت باقی می‌ماند.

در روش فعلی، توکن‌ها بر اساس مقدار confidence به صورت تدریجی commit می‌شوند. یک ایده‌ی بهبود، اضافه کردن یک مرحله‌ی پالایش یا بازبینی پس از چند بلوک است؛ به این صورت که توکن‌هایی که در مراحل اولیه تثبیت شده‌اند اما confidence پایینی داشته‌اند، دوباره مورد ارزیابی قرار گیرند. این مرحله می‌تواند مانند یک refinement pass عمل کند و خطاهای زنجیره‌ای ایجادشده در بلوک‌های ابتدایی را اصلاح کند. انتظار می‌رود این تغییر باعث افزایش پایداری مدل و کاهش خطاهای نحوی یا معنایی شود، به‌ویژه در کوئری‌های طولانی‌تر، بدون آنکه افزایش زمان تولید بسیار چشمگیر باشد.

در برخی نمونه‌ها، به‌ویژه زمانی که پاسخ کوتاه است، مدل ممکن است دچار overfitting شود یا خروجی

ناقص تولید کند. می‌توان یک جمله‌ی $loss$ کمکی تعریف کرد که طول خروجی تولیدشده را با طول SQL مرجع مقایسه کند یا حداقل درصد مشخصی از ماسک شدن را تضمین نماید. همچنین می‌توان از یک جریمه‌ی ملایم برای پاسخ‌های بسیار کوتاه یا بسیار طولانی استفاده کرد. این نوع $regularization$ انتظار می‌رود باعث تولید پاسخ‌های کامل‌تر و سازگارتر شود و از خروجی‌های ناقص جلوگیری کند، در حالی که تأثیر منفی محسوسی بر سرعت آموزش نخواهد داشت.

در حال حاضر، انتخاب توکن‌ها در هر موقعیت می‌تواند به صورت $greedy$ یا $sampling$ انجام شود. یک ایده‌ی توسعه‌ای، نگه داشتن چند مسیر محتمل در هر بلوک با استفاده از $beam search$ است. در این حالت، در هر مرحله چند دنباله‌ی برتر بر اساس مجموع $confidence$ و میزان سازگاری با $schema$ حفظ می‌شوند. این کار می‌تواند احتمال انتخاب ساختارهای نحوی صحیح‌تر را افزایش دهد و خطاهای مرتبط با نام جدول یا ستون را کاهش دهد. هرچند زمان تولید کمی افزایش می‌یابد، اما برای کوئری‌های کوتاه تا متوسط این افزایش قابل مدیریت است و در مقابل، کیفیت نهایی می‌تواند به طور محسوسی بهبود یابد.

به جای وارد کردن $schema$ به صورت متن خام، می‌توان بازنمایی‌های ساختاری‌تری طراحی کرد؛ برای مثال، ساخت $embedding$ هایی که روابط بین جدول‌ها، کلیدهای خارجی و ستون‌ها را به صورت گرافی مدل‌سازی کنند. این بازنمایی می‌تواند به مدل کمک کند وابستگی‌های بین اجزای پایگاه داده را بهتر درک کند و در نتیجه خطاهای مربوط به انتخاب نام ستون یا جدول کاهش یابد. انتظار می‌رود این تغییر به‌ویژه در کوئری‌های پیچیده با چندین $JOIN$ باعث افزایش دقت و کاهش تولید خروجی‌های نامعتبر شود، در حالی که افزایش مصرف حافظه یا زمان محاسبه محدود و قابل کنترل خواهد بود.